Unit 11 Exercises

4SC000 Optimal Control and Reinforcement Learning

Duarte Antunes

These exercises accompany Unit 11 and its slides, with companion Jupyter notebooks.

> ⚠ Note on the transition matrices
>
> In Problems 11.1 and 11.2 the transition matrices are defined by
>
> $$P_{ji}(\ell) = \text{Prob}[x_{k+1} = j \mid x_k = i,\ u_k = \ell],$$
>
> so the entry in row j, column i is the probability of moving **from** i **to** j, and each **column** of $P(\ell)$ sums to one. This is the transpose of the convention $p_{ij}(u)$ used in the notes and slides. In the policy evaluation step
>
> $$J_\mu(i) = g(i, \mu(i)) + \alpha \sum_j p_{ij}(\mu(i)) J_\mu(j),$$
>
> the sum is the inner product of the i-th **column** of $P(\mu(i))$ with the vector of values — not the i-th row. Equivalently, in matrix form the evaluation step is $J_\mu = g_\mu + \alpha P_\mu^\top J_\mu$, with the transpose. Getting this backwards is the commonest way to lose these problems.

Core exercises

Problem 11.1 Consider the following discounted cost optimal control problem

$$\min_{u_k = \mu_k(x_k)} \sum_{k=0}^{\infty} \mathbb{E}\left[\alpha^k g(x_k, u_k)\right], \qquad \text{Prob}[x_{k+1} = j \mid x_k = i,\ u_k = \ell] = P_{ji}(\ell),$$

where $\alpha = 0.8$ and $g(i, \ell) = |i| + |\ell|$ for every $i \in \{1, 2\}$ and $\ell \in \{1, 2\}$, and

$$P(1) = \begin{bmatrix} 0.7 & 0.4 \\ 0.3 & 0.6 \end{bmatrix}, \qquad P(2) = \begin{bmatrix} 0.3 & 0.6 \\ 0.7 & 0.4 \end{bmatrix}.$$

Compute the optimal policy using policy iteration with initial policy $\mu(1) = 2$, $\mu(2) = 2$.

> ℹ Solution
>
> Alternate policy evaluation and policy improvement until the improvement step returns the policy it was given. There are only four policies here, so the loop is short enough to do by hand.
> **Policy evaluation of $\mu^1 = (2, 2)$.** With the policy fixed there is no minimisation left, only a linear system. The costs are $g(1, 2) = 3$ and $g(2, 2) = 4$, and the transition matrix of the closed loop is $P(2)$:
>
> $$\begin{bmatrix} J_{\mu^1}(1) \\ J_{\mu^1}(2) \end{bmatrix} = \begin{bmatrix} 1+2 \\ 2+2 \end{bmatrix} + 0.8\, P(2)^\top \begin{bmatrix} J_{\mu^1}(1) \\ J_{\mu^1}(2) \end{bmatrix} \quad \Longrightarrow \quad \begin{bmatrix} J_{\mu^1}(1) \\ J_{\mu^1}(2) \end{bmatrix} = \begin{bmatrix} 17.2581 \\ 18.0645 \end{bmatrix}.$$

Policy improvement. Compute the Q-factors against these values. The ones for the current action are free, $Q^1(i,\mu^1(i)) = J_{\mu^1}(i)$, so only the alternative needs work:

$$Q^1(1,1) = 2{+}0.8(0.7J_{\mu^1}(1){+}0.3J_{\mu^1}(2)) = 16.0000, \qquad Q^1(2,1) = 3{+}0.8(0.4J_{\mu^1}(1){+}0.6J_{\mu^1}(2)) = 17.1935.$$

Both beat 17.2581 and 18.0645, so

$$\mu^2(i) = \arg\min_{u\in\{1,2\}} Q^1(i,u) = 1, \qquad i \in \{1,2\}.$$

Policy evaluation of $\mu^2 = (1,1)$. Now $g(1,1) = 2$, $g(2,1) = 3$ and the closed loop is $P(1)$:

$$\begin{bmatrix} J_{\mu^2}(1) \\ J_{\mu^2}(2) \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \end{bmatrix} + 0.8\, P(1)^\top \begin{bmatrix} J_{\mu^2}(1) \\ J_{\mu^2}(2) \end{bmatrix} \quad\Longrightarrow\quad \begin{bmatrix} J_{\mu^2}(1) \\ J_{\mu^2}(2) \end{bmatrix} = \begin{bmatrix} 11.5789 \\ 12.8947 \end{bmatrix}.$$

Both values have dropped by about a third — the improvement step delivered a genuinely better policy, not a marginal one.

Policy improvement. Again only the alternative action needs computing:

$$Q^2(1,2) = 3{+}0.8(0.3J_{\mu^2}(1){+}0.7J_{\mu^2}(2)) = 13.0000, \qquad Q^2(2,2) = 4{+}0.8(0.6J_{\mu^2}(1){+}0.4J_{\mu^2}(2)) = 13.6842,$$

against $Q^2(1,1) = 11.5789$ and $Q^2(2,1) = 12.8947$. The minimisers are unchanged,

$$\mu^3(i) = \arg\min_{u\in\{1,2\}} Q^2(i,u) = 1 = \mu^2(i),$$

so the policy cannot be improved and is therefore optimal: $\mu^\star(1) = \mu^\star(2) = 1$, with optimal cost $J^\star = (11.5789,\ 12.8947)$.

Two rounds, and the stopping test is exact rather than a tolerance: the improvement step returned its own input. That is the structural advantage of policy iteration over value iteration, which would approach these same numbers from below without ever certifying that it had arrived.

Problem 11.2 Consider the following finite-horizon optimal control problem

$$\min_{u_k=\mu_k(x_k)} \mathbb{E}\left[\sum_{k=0}^{1} g(x_k,u_k)\right], \qquad \text{Prob}[x_{k+1} = j \mid x_k = i,\ u_k = \ell] = P_{ji}(\ell),$$

where $g(i,\ell) = |i| + 2|\ell|$ for every $i \in \{1,2\}$ and $\ell \in \{1,2\}$, and

$$P(1) = \begin{bmatrix} 0.5 & 0.2 \\ 0.5 & 0.8 \end{bmatrix}, \qquad P(2) = \begin{bmatrix} 0.3 & 0.5 \\ 0.7 & 0.5 \end{bmatrix}.$$

Compute the optimal policy using policy iteration with initial policy $\mu_k(1) = 1$, $\mu_k(2) = 2$ for every $k \in \{0,1\}$.

i Solution

The problem could be turned into a stochastic shortest path problem by augmenting the state with the clock, as in Unit 10's Problem 10.1, and then run through infinite-horizon policy iteration. The simpler route is to work directly in the finite horizon: a policy is now a pair of maps, one per stage, and policy evaluation is a backward sweep rather than a linear solve.

Policy evaluation of μ^0. Evaluation is dynamic programming with only one action available per state and stage — the one the policy prescribes — so no minimisation is involved, only propagation. There is no terminal cost, so at the last stage

$$J_1^0(1) = g(1, \mu_1(1)) = 1 + 2(1) = 3, \qquad J_1^0(2) = g(2, \mu_1(2)) = 2 + 2(2) = 6,$$

and then, reading the columns of the appropriate matrix for each state,

$$J_0^0(1) = 3 + 0.5(3) + 0.5(6) = 7.5, \qquad J_0^0(2) = 6 + 0.5(3) + 0.5(6) = 10.5.$$

Policy improvement. At stage $k = 1$ the Q-factors involve no future, so $Q_1^0(1,1) = J_1^0(1) = 3$, $Q_1^0(2,2) = J_1^0(2) = 6$, and

$$Q_1^0(1,2) = 1 + 2(2) = 5, \qquad Q_1^0(2,1) = 2 + 2(1) = 4.$$

The improved policy at the last stage is $\mu_1^1(1) = 1$, $\mu_1^1(2) = 1$. At stage $k = 0$, with $Q_0^0(1,1) = 7.5$ and $Q_0^0(2,2) = 10.5$,

$$Q_0^0(1,2) = 5 + 0.3(3) + 0.7(6) = 10.1, \qquad Q_0^0(2,1) = 4 + 0.2(3) + 0.8(6) = 9.4,$$

so $\mu_0^1(1) = 1$, $\mu_0^1(2) = 1$. The policy changed at state 2 in both stages.

Policy evaluation of μ^1. At the last stage the values are already computed — they are the Q-factors of the chosen actions:

$$J_1^1(1) = Q_1^0(1,1) = 3, \qquad J_1^1(2) = Q_1^0(2,1) = 4.$$

At stage 0,

$$J_0^1(1) = 3 + 0.5(3) + 0.5(4) = 6.5, \qquad J_0^1(2) = 4 + 0.2(3) + 0.8(4) = 7.8.$$

Policy improvement. The stage-1 policy cannot change: its Q-factors depend only on the (absent) terminal cost, which has not moved. The stage-0 policy can, because J_1 has. With $Q_0^1(1,1) = J_0^1(1) = 6.5$ and $Q_0^1(2,1) = J_0^1(2) = 7.8$,

$$Q_0^1(1,2) = 5 + 0.3(3) + 0.7(4) = 8.7, \qquad Q_0^1(2,2) = 6 + 0.5(3) + 0.5(4) = 9.5.$$

The minimisers are unchanged, so the policy is optimal:

$$\mu_0^\star(1) = \mu_0^\star(2) = \mu_1^\star(1) = \mu_1^\star(2) = 1,$$

with $J_0^\star = (6.5,\ 7.8)$ and $J_1^\star = (3,\ 4)$.

Worth noting what the finite horizon does to the loop. Policy iteration converges here in two rounds, where plain backward dynamic programming would have produced the same answer in *one* sweep. Policy iteration is not the method of choice for finite-horizon problems; the exercise is here because the two steps are easiest to see in a setting where both can be done exactly by hand, and because the way information propagates — the last stage settling first, then freezing — is the pattern simulation-based policy iteration will have to reproduce without a model.

Problem 11.3 Consider an inventory control problem where the number of items of a given product is described by

$$x_{k+1} = \max\{x_k + u_k - d_k, 0\}, \qquad k \in \{0, 1, \dots\},$$

where $x_k \in \{0, \dots, N\}$, $u_k \in \{0, \dots, N - x_k\}$ and d_k denote the number of items, the supply and the demand at time k, and N denotes the capacity. Suppose $N = 2$ and that the demand is uncertain with $\text{Prob}[d_k = 0] = \text{Prob}[d_k = 1] = \text{Prob}[d_k = 2] = 1/3$ for every k. The objective is to find a policy minimising

$$\mathbb{E}\left[\sum_{k=0}^{\infty} \alpha^k (c_1(x_k) + c_2(u_k) - p \min\{x_k + u_k, d_k\})\right]$$

for $\alpha = 0.9$, where $c_1(i) = i$ is the storage cost, $c_2(j) = 5j + 0.9$ for $j > 0$ and $c_2(0) = 0$ is the cost of ordering j items, and $p = 9$ is the selling price per item.

(i) Run one step of policy evaluation, i.e. compute the cost function of the initial policy $\mu_0(i) = 0$ for every $i \in \{0, 1, \dots, N\}$,

$$J_{\mu_0}(i) = \mathbb{E}\left[\sum_{k=0}^{\infty} \alpha^k (c_1(x_k) + c_2(\mu_0(x_k)) - p \min\{x_k + \mu_0(x_k), d_k\}) \;\middle|\; x_0 = i\right].$$

(ii) Run one step of policy improvement, obtaining an improved policy $\mu_1(i)$, $i \in \{0, 1, 2\}$.

Solution

(i) Policy evaluation. With the policy fixed at "order nothing" the cost function solves the linear system

$$J_{\mu_0}(i) = \mathbb{E}[c_1(i) + c_2(\mu_0(i)) - p \min\{i + \mu_0(i), d_k\} + \alpha J_{\mu_0}(\max\{i + \mu_0(i) - d_k, 0\})], \qquad i \in \{0, 1, 2\}.$$

Two ingredients. The expected one-step cost under μ_0: nothing is ordered, so $c_2 = 0$ and

$$\mathbb{E}[g(0, 0, d)] = 0, \qquad \mathbb{E}[g(1, 0, d)] = 1 - 9\left(\tfrac{0+1+1}{3}\right) = -5, \qquad \mathbb{E}[g(2, 0, d)] = 2 - 9\left(\tfrac{0+1+2}{3}\right) = -7.$$

And the closed-loop transition matrix, with $x' = \max\{i - d, 0\}$: from stock 0 the shop stays at 0; from 1 it goes to 1 with probability $1/3$ and to 0 with probability $2/3$; from 2 it goes to each of $0, 1, 2$ with probability $1/3$. Hence

$$\begin{bmatrix} J_{\mu_0}(0) \\ J_{\mu_0}(1) \\ J_{\mu_0}(2) \end{bmatrix} = \begin{bmatrix} 0 \\ -5 \\ -7 \end{bmatrix} + 0.9 \begin{bmatrix} 1 & 0 & 0 \\ 2/3 & 1/3 & 0 \\ 1/3 & 1/3 & 1/3 \end{bmatrix} \begin{bmatrix} J_{\mu_0}(0) \\ J_{\mu_0}(1) \\ J_{\mu_0}(2) \end{bmatrix} \quad \Longrightarrow \quad \begin{bmatrix} J_{\mu_0}(0) \\ J_{\mu_0}(1) \\ J_{\mu_0}(2) \end{bmatrix} = \begin{bmatrix} 0 \\ -7.1429 \\ -13.0612 \end{bmatrix}.$$

The system is lower triangular, so it can be solved by hand from the top down: $J(0) = 0.9J(0)$ gives $J(0) = 0$ — an empty shop that never orders stays empty and earns nothing forever — then $0.7J(1) = -5$ and $0.7J(2) = -7 + 0.3J(1)$.

(ii) Policy improvement. Compute

$$\mu_1(i) = \arg \min_{u \in \{0, \dots, 2-i\}} \mathbb{E}[g(i, u, d_k) + \alpha J_{\mu_0}(\max\{i + u - d_k, 0\})].$$

At $i = 2$ there is only one admissible control, so $\mu_1(2) = 0$.
At $i = 1$ there are two. For $u = 0$:

	$d_k = 0$	$d_k = 1$	$d_k = 2$
	$1 + 0.9(-7.1429) = -5.4286$	$1 - 9 + 0 = -8$	$1 - 9 + 0 = -8$

with expected value -7.1429 — which is $J_{\mu_0}(1)$ again, as it must be. For $u = 1$:

$d_k = 0$	$d_k = 1$	$d_k = 2$
$1 + 5.9 + 0.9(-13.0612) = -4.8551$	$1 + 5.9 - 9 + 0.9(-7.1429) = -8.5286$	$1 + 5.9 - 18 + 0 = -11.1$

with expected value -8.1612. So $\mu_1(1) = 1$.
At $i = 0$ there are three. For $u = 0$ every outcome costs 0. For $u = 1$:

$d_k = 0$	$d_k = 1$	$d_k = 2$
$5.9 + 0.9(-7.1429) = -0.5286$	$5.9 - 9 = -3.1$	$5.9 - 9 = -3.1$

and for $u = 2$:

$d_k = 0$	$d_k = 1$	$d_k = 2$
$10.9 + 0.9(-13.0612) = -0.8551$	$10.9 - 9 + 0.9(-7.1429) = -4.5286$	$10.9 - 18 = -7.1$

The expected costs of $u = 0, 1, 2$ are 0, -2.2429 and -4.1612, so $\mu_1(0) = 2$. Collecting,

$$(\mu_1(0), \mu_1(1), \mu_1(2)) = (2, 1, 0).$$

The pattern is the natural one: fill the shop when it is empty, top it up by one when it holds one, and do nothing when it is full.

Where the loop goes next

The exercise stops after one round, but it is worth knowing that one more round finishes the job. Evaluating $(2, 1, 0)$ gives $J = (-19.70, -23.70, -28.60)$, whose improvement step returns $(2, 0, 0)$; evaluating *that* gives $J = (-20.39, -24.62, -29.29)$ and the improvement step returns $(2, 0, 0)$ again. So the optimal policy is "order two when empty, otherwise nothing", reached in three policy evaluations.

Note what changed between rounds: $\mu_1(1) = 1$ was an improvement on μ_0, but once the shop's value is computed under a policy that actually orders, holding one item becomes good enough on its own and the top-up is not worth its 0.9 transaction cost. An improvement step is greedy with respect to the *current* policy's value, and being an improvement is not the same as being optimal.

Problem 11.4 Repeat Unit 10's Problem 10.4 — Q-learning by hand on the ten-step data table for the three-stage inventory problem — but now with SARSA, i.e. with the update

$$Q_{\ell+1}(k, x_k, u_k) = (1 - \gamma^\ell) Q_\ell(k, x_k, u_k) + \gamma^\ell (g(x_k, u_k, d_k) + Q_\ell(k + 1, x_{k+1}, u_{k+1})), \qquad k \neq h - 1,$$

the terminal update and all the problem data being unchanged. Recall $h = 3$, $N = 2$, $\gamma^\ell = 0.5$, $g(x_k, u_k, d_k) = x_k + c_2(u_k) - 9 \min\{x_k + u_k, d_k\}$ with $c_2(j) = 5j + 0.9$ for $j > 0$, $g_h(i) = -4i$, and the data

ℓ	1	2	3	4	5	6	7	8	9	10
k	0	1	2	3	0	1	2	3	0	1
x_k	1	0	2	2	0	0	1	0	0	1
u_k	1	2	0	–	1	2	1	–	1	1
d_k	2	0	0	–	1	1	2	–	0	1

Solution

One symbol changes: the $\min_v Q_\ell(k+1, x_{k+1}, v)$ of Q-learning becomes $Q_\ell(k+1, x_{k+1}, u_{k+1})$, the estimate for the action the behaviour policy *actually took* at the next step. Everything else — step size, running costs, terminal updates — is as in Problem 10.4.

That one symbol has a consequence for the data, and it is the point of the exercise. Q-learning's update needs only (x_k, u_k, d_k, x_{k+1}), all of which the table provides. SARSA needs u_{k+1} as well, so the next row of the table is part of the current update. The table has ten rows, so the last update cannot be completed from it — more on that below.

Table 1: The SARSA pass. The bootstrap column is what replaces Q-learning's minimum.

ℓ	k	x_k	u_k	d_k	g	x_{k+1}	bootstrap	$Q_{\ell+1}(k, x_k, u_k)$
1	0	1	1	2	-11.1	0	$Q(1,0,2) = 0$	$Q(0,1,1) = -5.55$
2	1	0	2	0	10.9	2	$Q(2,2,0) = 0$	$Q(1,0,2) = 5.45$
3	2	2	0	0	2	2	$g_3(2) = -8$	$Q(2,2,0) = -3$
4	3	2	–	–	–	–	terminal	no update
5	0	0	1	1	-3.1	0	$Q(1,0,2) = 5.45$	$Q(0,0,1) = 1.175$
6	1	0	2	1	1.9	1	$Q(2,1,1) = 0$	$Q(1,0,2) = 3.675$
7	2	1	1	2	-11.1	0	$g_3(0) = 0$	$Q(2,1,1) = -5.55$
8	3	0	–	–	–	–	terminal	no update
9	0	0	1	0	5.9	1	$Q(1,1,1) = 0$	$Q(0,0,1) = 3.5375$
10	1	1	1	1	-2.1	1	u_{11} unknown	see below

Three of the updates differ numerically from Problem 10.4, and each difference is instructive.
At $\ell = 5$ the successor is state 0 at stage 1, where the action actually taken next (row $\ell = 6$) is $u = 2$, and $Q(1, 0, 2)$ has already been raised to 5.45 by $\ell = 2$. SARSA must use that 5.45:

$$Q_6(0, 0, 1) = 0.5(-3.1) + 0.5(5.45) = 1.175,$$

against Q-learning's -1.55. Q-learning took the minimum over $v \in \{0, 1, 2\}$ and found two untried actions still sitting at 0, so it bootstrapped on 0 and ignored the one piece of evidence it had. SARSA cannot ignore it: the behaviour policy chose $u = 2$, and $u = 2$ is what gets evaluated. The gap of 2.725 between the two answers is the whole off-policy/on-policy distinction, in one update.
At $\ell = 9$ the successor is state 1 at stage 1, where row $\ell = 10$ takes $u = 1$ and $Q(1, 1, 1)$ is still 0:

$$Q_{10}(0, 0, 1) = 0.5(1.175) + 0.5(5.9 + 0) = 3.5375,$$

against Q-learning's 2.175 — this time the difference comes from the estimate the two algorithms carry into the update, not from the bootstrap itself.
The last update cannot be done. At $\ell = 10$ the successor is $x_{11} = 1 + 1 - 1 = 1$ at stage 2, and SARSA needs u_{11}, the action taken there. The table stops at $\ell = 10$, so it is not given. Both branches:

$$u_{11} = 0: \quad Q_{11}(1, 1, 1) = 0.5(-2.1) + 0.5\, Q_{10}(2, 1, 0) = 0.5(-2.1 + 0) = -1.05,$$

$$u_{11} = 1: \quad Q_{11}(1, 1, 1) = 0.5(-2.1) + 0.5\, Q_{10}(2, 1, 1) = 0.5(-2.1 - 5.55) = -3.825.$$

The second coincides with Q-learning's answer, because $Q(2, 1, 1) = -5.55$ happens to be the minimum over the two admissible controls at that state. That is the general relationship: SARSA agrees with Q-learning exactly when the behaviour policy happens to pick the greedy action, and an ϵ-greedy behaviour policy does so most of the time — which is why the two produce similar policies in practice, and why `A_11_FrozenLakeQLearningSARSA.ipynb` has to look at the arrow grids rather than the win rates to tell them apart.
So the estimates after nine iterations, which is as far as the data takes SARSA, are

$$\begin{array}{lll} Q_{10}(0, 0, 1) = 3.5375 & Q_{10}(0, 1, 1) = -5.55 & Q_{10}(1, 0, 2) = 3.675 \\ Q_{10}(2, 1, 1) = -5.55 & Q_{10}(2, 2, 0) = -3 & Q_{10}(1, 1, 1) = 0 \end{array}$$

with every other triple zero.

Why the data table is one column short

The missing u_{11} is not a defect in the problem; it is the cleanest possible statement of what separates the two algorithms. Q-learning learns the Q-factors of the *optimal* policy from data generated by any policy, so a transition (x_k, u_k, g, x_{k+1}) is a self-contained training example and the data set is a *set*. SARSA learns the Q-factors of the policy that *generated* the data, so its training example is $(x_k, u_k, g, x_{k+1}, u_{k+1})$ — the quintuple the algorithm is named after — and the data set is a *sequence* that cannot be cut anywhere without losing one example.

Problem 11.5 The following data was obtained for a discounted cost infinite horizon problem with 3 states and discount factor $\alpha = 0.9$.

time k	1	2	3	4	5	6	7	8	9	10	11
state x	1	2	3	1	2	1	1	1	2	1	1
cost $g(x_k)$	1	4	9	1	4	1	1	1	4	1	1

Compute value estimates $\hat{J}(1)$, $\hat{J}(2)$, $\hat{J}(3)$, assuming these are initially zero, with TD and with Monte-Carlo, using $\gamma = 0.9$:

- TD: $\hat{J}(x_k) \leftarrow \hat{J}(x_k) + \gamma(g(x_k) + \alpha\hat{J}(x_{k+1}) - \hat{J}(x_k))$
- Monte-Carlo: $\hat{J}(x_k) \leftarrow \hat{J}(x_k) + \gamma(R_k - \hat{J}(x_k))$, with $R_k = \sum_{\ell=k}^{T} \alpha^{\ell-k} g(x_\ell)$ and $T = 11$

Note that the Monte-Carlo method approximates the infinite-horizon returns by finite-horizon ones, truncated at the end of the data.

i Solution

TD. Each sample updates one state, using the current estimate of its successor. The estimates after processing each time step:

Table 2: TD estimates. The last row is the answer; $k = 11$ has no successor, so there is no eleventh update.

after k	$\hat{J}(1)$	$\hat{J}(2)$	$\hat{J}(3)$
1	0.9000	0	0
2	0.9000	3.6000	0
3	0.9000	3.6000	8.8290
4	3.9060	3.6000	8.8290
5	3.9060	7.1239	8.8290
6	4.4545	7.1239	8.8290
7	4.9536	7.1239	8.8290
8	7.1657	7.1239	8.8290
9	7.1657	10.1166	8.8290
10	**7.4208**	**10.1166**	**8.8290**

This is a direct application of the formula. For instance at $k = 3$, with $x_3 = 3$, $x_4 = 1$ and $\hat{J}(1) = 0.9$ at that moment,

$$\hat{J}(3) \leftarrow \underbrace{\hat{J}(3)}_{=0} + 0.9(9 + 0.9\,\underbrace{\hat{J}(1)}_{=0.9} - \underbrace{\hat{J}(3)}_{=0}) = 0.9(9.81) = 8.8290.$$

State 3 is visited once, so this single value is also TD's final answer for it — and it was computed when $\hat{J}(1)$ was still 0.9 rather than its final 7.42. TD's estimates are only as good as the estimates they bootstrapped on at the time.

Monte-Carlo. First the truncated returns, computed backwards from $R_{11} = g(x_{11}) = 1$ via $R_k = g(x_k) + \alpha R_{k+1}$:

Table 3: Truncated returns.

k	1	2	3	4	5	6	7	8	9	10	11
R_k	19.3016	20.3351	18.1501	10.1668	10.1853	6.8726	6.5251	6.1390	5.7100	1.9000	1.0000

Then the updates:

Table 4: Monte-Carlo estimates.

after k	$\hat{J}(1)$	$\hat{J}(2)$	$\hat{J}(3)$
1	17.3714	0	0
2	17.3714	18.3016	0
3	17.3714	18.3016	16.3351
4	10.8873	18.3016	16.3351
5	10.8873	10.9970	16.3351
6	7.2741	10.9970	16.3351
7	6.6000	10.9970	16.3351
8	6.1851	10.9970	16.3351
9	6.1851	6.2387	16.3351
10	**2.3285**	6.2387	16.3351
11	1.1329	**6.2387**	**16.3351**

Again a direct application, $\hat{J}(x_k) \leftarrow \hat{J}(x_k) + \gamma(R_k - \hat{J}(x_k))$. For example at $k = 9$, where $R_9 = 4 + \alpha + \alpha^2 = 5.71$,

$$\hat{J}(2) \leftarrow \underbrace{\hat{J}(2)}_{=10.9970} + 0.9(5.71 - \underbrace{\hat{J}(2)}_{=10.9970}) = 6.2387.$$

The solutions manual stops the Monte-Carlo pass at $k = 10$ and reports $\hat{J}(1) = 2.3285$. Applying the stated rule to the last sample as well, with $R_{11} = 1$, gives $\hat{J}(1) = 1.1329$. Either is a defensible reading of the recipe — the question is whether a return truncated to a single term counts as a sample — and nothing below depends on which is taken.

Two things worth reading off these tables.

The estimates are not averages. With $\gamma = 0.9$ each update discards 90% of what was there, so the final number for a state is dominated by the *last* sample that touched it. Monte-Carlo's $\hat{J}(1)$ falls monotonically, $17.37 \to 10.89 \to 7.27 \to 6.60 \to 6.19 \to 2.33$, not because state 1 is getting cheaper but because later returns are truncated harder: $R_{10} = 1.9$ contains two terms where R_1 contains eleven. A step size this large is appropriate for a *demonstration* of the two updates and wrong for estimation; the convergence theorem asks for $\sum \gamma^\ell = \infty$ with $\sum(\gamma^\ell)^2 < \infty$, and a constant 0.9 satisfies neither.

The two methods disagree, and systematically. TD ends at (7.42, 10.12, 8.83) and Monte-Carlo at (2.33, 6.24, 16.34). TD's numbers are the more sensible ones here, for a reason specific to this data: TD bootstraps on a stored estimate, which carries information from the whole history, while Monte-Carlo's return stops dead at $k = 11$ and therefore systematically *underestimates* an infinite-horizon cost — badly, for the samples near the end. Monte-Carlo pays no bias for bootstrapping on wrong values, but it does pay for truncation, and with $\alpha = 0.9$ an eleven-step window captures only $1 - 0.9^{11} \approx 69\%$ of the discount mass. The usual summary — TD biased but low variance, MC unbiased but high variance — assumes complete episodes, which this data does not provide.

Problem 11.6 Show that for an infinite-horizon discounted problem the Monte-Carlo error used for the update can be computed as a weighted sum of TD errors, provided TD does not update these values along the trajectory, i.e. show that

$$R_k - \hat{J}(x_k) = \sum_{\ell=k}^{\infty} \alpha^{\ell-k} \delta(x_\ell),$$

where

$$R_k = \sum_{\ell=k}^{\infty} \alpha^{\ell-k} g(x_\ell), \qquad \delta(x_\ell) = g(x_\ell) + \alpha \hat{J}(x_{\ell+1}) - \hat{J}(x_\ell).$$

Solution

Substitute the definition of δ and split the sum into three:

$$\begin{aligned}\sum_{\ell=k}^{\infty} \alpha^{\ell-k} \delta(x_\ell) &= \sum_{\ell=k}^{\infty} \alpha^{\ell-k} (g(x_\ell) + \alpha \hat{J}(x_{\ell+1}) - \hat{J}(x_\ell)) \\ &= \underbrace{\sum_{\ell=k}^{\infty} \alpha^{\ell-k} g(x_\ell)}_{=R_k} + \sum_{\ell=k}^{\infty} \alpha^{\ell-k+1} \hat{J}(x_{\ell+1}) - \sum_{\ell=k}^{\infty} \alpha^{\ell-k} \hat{J}(x_\ell).\end{aligned}$$

Reindex the middle sum with $m = \ell + 1$, so that $\alpha^{\ell-k+1} = \alpha^{m-k}$:

$$\sum_{\ell=k}^{\infty} \alpha^{\ell-k+1} \hat{J}(x_{\ell+1}) = \sum_{m=k+1}^{\infty} \alpha^{m-k} \hat{J}(x_m).$$

The last two sums are now the same series except that the first starts at $m = k+1$ and the second at $\ell = k$, so everything cancels except the single term $\alpha^0 \hat{J}(x_k)$ of the second:

$$\sum_{m=k+1}^{\infty} \alpha^{m-k} \hat{J}(x_m) - \sum_{\ell=k}^{\infty} \alpha^{\ell-k} \hat{J}(x_\ell) = -\hat{J}(x_k).$$

Hence $\sum_{\ell=k}^{\infty} \alpha^{\ell-k} \delta(x_\ell) = R_k - \hat{J}(x_k)$, which is the claim. (The telescoping is legitimate provided the series converge absolutely, which they do when g and $\hat{J}$ are bounded and $\alpha < 1$.)

Why the proviso matters. The cancellation uses the *same* function $\hat{J}$ at every ℓ — the $\hat{J}(x_m)$ coming out of one TD error must be the $\hat{J}(x_m)$ going into the next. If TD updates $\hat{J}$ as it walks the trajectory, as it does in Problem 11.5, the two are different numbers and the identity holds only approximately.

What it buys. The identity says Monte-Carlo and TD are the two ends of one family rather than two unrelated algorithms: a Monte-Carlo update is the sum of all the TD errors along the trajectory, discounted, and a TD update is its first term. Truncating the sum after n terms gives n-step TD; weighting the truncations geometrically gives TD(λ), with $\lambda = 0$ recovering TD and $\lambda = 1$ recovering Monte-Carlo. It also explains the bias/variance trade-off structurally: Monte-Carlo accumulates every sampled δ and therefore every sample's noise, while TD keeps only the first and absorbs the rest into a stored estimate.

Problem 11.7 Consider a random variable $X \in \{x_1, \ldots, x_m\}$ distributed according to $p_i = \text{Prob}[X = x_i]$. Suppose we sample it instead according to a different distribution q_i. The distributions are

x_i	1	2	3	4	5
p_i	0.03	0.15	0.7	0.07	0.05
q_i	0.3	0.2	0.1	0.2	0.2

Suppose the following data was obtained by sampling with distribution q. Compute mean estimates of X with and without importance sampling.

sample	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
value	1	1	5	4	3	2	1	1	2	4	5	5	3	4	1	1	3	4	1	2

i Solution

The twenty samples contain the value 1 seven times, 2 three times, 3 three times, 4 four times and 5 three times.

Without importance sampling, just average them:

$$\frac{7(1)+3(2)+3(3)+4(4)+3(5)}{20}=\frac{53}{20}=2.65.$$

This estimates $\mathbb{E}_q[X]=2.8$, not the quantity asked for.

With importance sampling, weight each sample by p_i/q_i:

x_i	1	2	3	4	5
p_i/q_i	0.1	0.75	7	0.35	0.25

$$\hat{\mathbb{E}}[X]=\frac{0.1(7)(1)+0.75(3)(2)+7(3)(3)+0.35(4)(4)+0.25(3)(5)}{20}=\frac{77.55}{20}=3.8775.$$

The true answer is $\mathbb{E}_p[X]=0.03(1)+0.15(2)+0.7(3)+0.07(4)+0.05(5)=2.96$, so the naive average is off by -0.31 and ordinary importance sampling by $+0.92$ — three times worse, in the opposite direction.

Why, and the fix. Look at the weights: q puts only 0.1 on the value 3, which carries 0.7 of p's mass, so every sampled 3 comes in with weight 7. Three 3s arrived in twenty draws, where p would have produced about fourteen; the weights are supposed to correct for exactly that, but with so few draws of the dominant value the correction is applied to a sample that barely contains it. The sum of the weights makes the problem visible:

$$\sum_{r=1}^{20}\frac{p_{i_r}}{q_{i_r}}=26.1 \quad \text{against} \quad N=20.$$

In expectation that sum is N (each weight has mean $\sum_i q_i(p_i/q_i)=1$), and here it overshoots by 31%. Dividing by its actual value instead of by N gives the **weighted** importance sampling estimate:

$$\hat{\mathbb{E}}_{\mathrm{w}}[X]=\frac{77.55}{26.1}=2.9713,$$

against the true 2.96. The same twenty samples, the same weights, a different normaliser, and essentially the right answer.

That is the trade recorded on the unit's slides. Ordinary importance sampling is unbiased and, on a small sample, can be far off. Weighted importance sampling is biased — the ratio of two random quantities is not the ratio of their means — and has much lower variance, which is usually the better deal. The effect is visible with real numbers in `A_11_BlackjackOnOffPolicy.ipynb`, where the two estimators are run side by side against a Q-factor known exactly.

Problem 11.8 Suppose we want to estimate the value of a deterministic policy for a finite horizon optimal control problem, using as behaviour policy an ϵ-soft version of it. For simplicity take $n=2$ states, $h=2$ decision stages and two decisions per state:

$$J_0(x_0)=g(x_0,u_0)+g(x_1,u_1)+g_2(x_2),$$

the model being the same at stages 0 and 1, characterised by $p_{i'i}(j)=\mathrm{Prob}[x_{k+1}=i' \mid x_k=i,\ u_k=j]$ for $i,i'\in\{1,2\}$, $j\in\{1,2\}$, $k\in\{0,1\}$. Suppose the initial state is 1 and the target policy is $\pi(i)=1$ for every i.

1. Explain how to compute the estimated value of $J_0(1)$ from data with importance sampling, writing its expression explicitly.

2. Can you find an alternative, simpler method that performs better — and in what sense better?

Solution

1. Importance sampling estimates $J_0(1)$ as

$$\hat{J}_0(1) = \frac{1}{s}\sum_{r=1}^{s} \rho_r R_r,$$

where s is the number of simulated trajectories, $R_r = g(x_0, u_0) + g(x_1, u_1) + g_2(x_2)$ is the return of trajectory r, and ρ_r is its importance ratio

$$\rho_r = \frac{\pi(u_0 \mid x_0)}{\mu(u_0 \mid x_0)} \cdot \frac{\pi(u_1 \mid x_1)}{\mu(u_1 \mid x_1)}.$$

Note that only the *policy* probabilities appear: the transition probabilities are identical under both policies and cancel, which is what makes the method usable without a model.
The target policy is deterministic, $\pi(1 \mid i) = 1$ and $\pi(2 \mid i) = 0$, and the behaviour policy is its ϵ-soft version, $\mu(1 \mid i) = 1 - \epsilon$ and $\mu(2 \mid i) = \epsilon$. So $\rho_r = 0$ unless the trajectory took $u_0 = 1$ *and* $u_1 = 1$, in which case

$$\rho_r = \frac{1}{(1-\epsilon)^2}.$$

Every trajectory that ever deviated from the target contributes nothing. Writing R_r^1 for the return of a trajectory with $u_0 = u_1 = 1$ and s^1 for the number of such trajectories,

$$\hat{J}_0(1) = \frac{1}{s}\sum_{r:\, u_0=u_1=1} \frac{1}{(1-\epsilon)^2} R_r^1 = \frac{1}{(1-\epsilon)^2} \cdot \frac{s^1}{s} \cdot \frac{1}{s^1} \sum_{r:\, u_0=u_1=1} R_r^1.$$

Since $\mathbb{E}[s^1] = s(1-\epsilon)^2$, the first two factors have expectation 1 and

$$\mathbb{E}[\hat{J}_0(1)] = \mathbb{E}[R_r^1] = J_0(1),$$

so the estimator is unbiased.
2. The displayed identity suggests its own improvement. The factor $1/(1-\epsilon)^2$ exists only to undo the thinning that discarded the deviating trajectories, and s^1/s is a random quantity whose mean is precisely $(1-\epsilon)^2$. Replace the two by nothing at all: keep the trajectories that followed the target policy and average them without weights,

$$\hat{J}_0^{\text{alt}}(1) = \frac{1}{s^1}\sum_{r:\, u_0=u_1=1} R_r^1.$$

This is still unbiased — conditional on s^1, each kept R_r^1 has mean $J_0(1)$ — and has strictly smaller variance, because it has removed a source of randomness that carried no information: the *number* of usable trajectories. The first estimator's value moves when s^1 happens to come out high or low; the second's does not.
In what sense better, and the general lesson. Better in variance at equal bias, from the same data, at lower cost. And note what $\hat{J}^{\text{alt}}$ is: since every non-zero weight here equals $1/(1-\epsilon)^2$, dividing by the sum of the weights instead of by s gives exactly this estimator. So for a *deterministic* target policy, weighted importance sampling degenerates to "discard the trajectories that deviated and average the rest", and no importance ratios need to be computed at all. The machinery of importance sampling earns its keep when the target policy is stochastic, or when the trajectories are long enough that discarding every imperfect one would leave nothing — which, with h stages and a deterministic target, happens at the rate $(1-\epsilon)^h$.

Additional exercises

Problem 11.9 Consider an inventory control problem where the number of items of a given product over h stages is described by

$$x_{k+1} = \max\{x_k + u_k - d_k, 0\}, \qquad k \in \{0, \dots, h-1\},$$

where $x_k \in \{0, \dots, N\}$, $u_k \in \{0, \dots, N - x_k\}$ and d_k denote the number of items, the supply and the demand at time k, and N denotes the capacity. The objective is to minimise

$$\sum_{k=0}^{h-1} \left(c_1(x_k) + c_2(u_k) - p \min\{x_k + u_k, d_k\}\right) + g_h(x_h),$$

where $c_1(i) = i$ is the storage cost, $c_2(j) = 5j + 0.9$ for $j > 0$ and $c_2(0) = 0$ is the cost of ordering j items, $p = 9$ is the selling price per item, and $g_h(i) = -4i$ is the terminal cost. Suppose $h = 3$, $N = 2$, that $d_0 = 1$ and $d_2 = 1$ are known, and that only the middle demand d_1 is uncertain, with $\text{Prob}[d_1 = 0] = \text{Prob}[d_1 = 1] = \text{Prob}[d_1 = 2] = 1/3$.

Run one step of policy evaluation, i.e. compute the cost-to-go of the initial policy $\mu_k^0(i) = 0$ for every k, i, and one step of policy improvement, obtaining an improved policy $\mu_k^1(x_k)$.

Solution

The finite-horizon problem could be converted to a shortest path problem by augmenting the state with the clock; as in Problem 11.2 it is simpler to work directly with the stage-indexed cost-to-go.

Policy evaluation. Evaluating μ^0 is dynamic programming with exactly one action available per state-stage pair — the one the policy prescribes — so there is no minimisation, only propagation:

$$J_k(x_k) = \mathbb{E}[g(x_k, \mu_k^0(x_k), d_k) + J_{k+1}(\max\{x_k + \mu_k^0(x_k) - d_k, 0\})].$$

With $\mu_k^0 \equiv 0$ this is

$$J_k(x_k) = \mathbb{E}[x_k - 9\min\{x_k, d_k\} + J_{k+1}(\max\{x_k - d_k, 0\})],$$

which, stage by stage with $d_0 = d_2 = 1$ and d_1 uniform, reads

$$J_2(x_2) = x_2 - 9\min\{x_2, 1\} + g_3(\max\{x_2 - 1, 0\}),$$

$$J_1(x_1) = x_1 + \tfrac{1}{3} \sum_{d \in \{0,1,2\}} (-9\min\{x_1, d\} + J_2(\max\{x_1 - d, 0\})), \qquad J_0(x_0) = x_0 - 9\min\{x_0, 1\} + J_1(\max\{x_0 - 1, 0\}).$$

Working backwards from $g_3 = (0, -4, -8)$:

Table 5: Cost-to-go of the do-nothing policy — the first lattice on the slides.

stock	$J_0^{\mu^0}$	$J_1^{\mu^0}$	$J_2^{\mu^0}$	g_3
0	0	0	0	0
1	−8	−7.6667	−8	−4
2	−14.6667	−13.3333	−11	−8

Three of these are worth doing in full. At stage 2, $J_2(1) = 1-9+g_3(0) = -8$ and $J_2(2) = 2-9+g_3(1) = -11$. At stage 1,

$$J_1(1) = 1 + \tfrac{1}{3}\Big(\underbrace{J_2(1)}_{-8} + \underbrace{(-9 + J_2(0))}_{-9} + \underbrace{(-9 + J_2(0))}_{-9} \Big) = 1 - \tfrac{26}{3} = -7.6667,$$

$$J_1(2) = 2 + \tfrac{1}{3}\Big(\underbrace{J_2(2)}_{-11} + \underbrace{(-9 + J_2(1))}_{-17} + \underbrace{(-18 + J_2(0))}_{-18} \Big) = 2 - \tfrac{46}{3} = -13.3333,$$

and at stage 0, $J_0(1) = 1 - 9 + J_1(0) = -8$ and $J_0(2) = 2 - 9 + J_1(1) = -7 - 7.6667 = -14.6667$. An empty shop that never orders has $J_k(0) = 0$ at every stage: it never holds anything, never sells anything, and never pays anything.

Policy improvement. Compute $\mu_k^1(x_k) = \arg\min_u \mathbb{E}[g(x_k, u, d_k) + J_{k+1}^{\mu^0}(\cdot)]$, one stage at a time. At $x_k = 2$ the only admissible control is 0 at every stage.

Stage $k = 2$, where the continuation is the terminal cost and $d_2 = 1$:

x_2	$u=0$	$u=1$	$u=2$	μ_2^1
0	0	−3.1	−2.1	1
1	−8	−6.1	—	0
2	−11	—	—	0

For instance $x_2 = 1$, $u = 1$ gives $1 + 5.9 - 9\min\{2, 1\} + g_3(1) = -6.1$, worse than the -8 of ordering nothing.
Stage $k = 1$, where the demand is random and the continuation is $J_2^{\mu^0}$. For $x_1 = 1$, $u = 0$:

$d_1 = 0$	$d_1 = 1$	$d_1 = 2$	mean
$1 - 8 = -7$	$1 - 9 + 0 = -8$	$1 - 9 + 0 = -8$	-7.6667

and for $u = 1$:

$d_1 = 0$	$d_1 = 1$	$d_1 = 2$	mean
$1 + 5.9 - 11 = -4.1$	$1 + 5.9 - 9 - 8 = -10.1$	$1 + 5.9 - 18 = -11.1$	-8.4333

so $\mu_1^1(1) = 1$. For $x_1 = 0$ the three options give 0, -2.7667 and -4.4333, so $\mu_1^1(0) = 2$.
Stage $k = 0$, with $d_0 = 1$ known and the continuation $J_1^{\mu^0}$:

x_0	$u = 0$	$u = 1$	$u = 2$	μ_0^1
0	0	-3.1	-5.7667	2
1	-8	-9.7667	—	1
2	-14.6667	—	—	0

Collecting the improved policy:

Table 6: The improved policy μ^1 — the second lattice on the slides.

stock	μ_0^1	μ_1^1	μ_2^1
0	2	2	1
1	1	1	0
2	0	0	0

One round is not enough here
Unlike Problem 11.1, where two rounds sufficed, μ^1 is **not** yet optimal. Exact backward dynamic programming on the same problem gives

$$J_0^\star = (-8.5667, -13.4667, -16.7333), \qquad J_1^\star = (-5.4667, -9.7333, -14.3667), \qquad J_2^\star = (-3.1, -8, -11),$$

with optimal policies $\mu_0^\star = (1, 0, 0)$, $\mu_1^\star = (2, 0, 0)$ and $\mu_2^\star = (1, 0, 0)$. Only the last stage agrees with μ^1, which is no accident: the last stage's Q-factors are computed against the exact terminal cost, so they are right the first time, while the earlier stages were greedy with respect to the do-nothing policy's values. Improvement propagates backwards one stage per round, which is why policy iteration on an h-stage problem can need up to h rounds to do what one backward sweep does directly. The exercise asks for one round; it is worth knowing that one round is a step, not an answer.

Problem 11.10 Suppose that we wish to sell a used car by finding an optimal policy for accepting offers. The car is exactly two years old, and the offer for cars of this model and age in the initial month of the sale follows a distribution modelled by a random variable w_0 taking values in $\{c_1, \dots, c_C\}$, with $c_i = 10000 + (i-1)1000$ euros and $C = 5$, and $\text{Prob}[w_0 = c_i] = p_i$, $\sum_{i=1}^{C} p_i = 1 - p_0$, where p_0 is the probability of no offer. Assume $p_1 = 0.05$, $p_2 = 0.2$, $p_3 = 0.3$, $p_4 = 0.3$, $p_5 = 0.1$. The car depreciates at a rate α every month after the initial one: if $\bar{x}_k$ is the offer at month k, then $\bar{x}_k/\alpha^k$ has the same distribution as w_0.

After ten years the residual value is almost zero, so consider

$$\sum_{k=0}^{\infty} \alpha^k g(x_k, u_k),$$

where $u_k \in \{0, 1\}$ is the decision to sell ($u_k = 1$) or not ($u_k = 0$), $x_k = \mathrm{N}$ if there is no offer, $x_k = T$ if an offer was already accepted, and otherwise $x_k = \bar{x}_k/\alpha^k$. The reward is $g(\mathrm{N}, u_k) = g(T, u_k) = 0$ and

$$g(x_k, 1) = x_k, \qquad g(x_k, 0) = 0.$$

Compute the optimal policy using policy iteration, starting from the policy "do not sell" in every state.

i Solution

Two things to settle before computing

The objective is a maximisation. As in Unit 10's Problem 10.5, the source prints a minimisation with $g(x_k, 1) = x_k$, which would make "never sell" optimal. One wants the discounted sale price to be as *large* as possible, so the problem is read here as maximising it.

The data is inconsistent as printed. The statement requires $\sum_i p_i = 1 - p_0$ with $p_0 = 0.01$, i.e. 0.99, but the five listed values sum to 0.95. The repair with the fewest changes is to take the five p_i as given and set $p_0 = 0.05$, which is what is used below. (A value of α is also never supplied; $\alpha = 0.99$ is used, the monthly figure the asset-selling scripts in `code/unit10/` and `code/unit11/` use. The last remark below says what changes if these choices are made differently.)

The structure first. Two observations — the same two as in Problem 10.5 — reduce the whole problem to a single scalar.

Selling at an offer x collects x and moves to the absorbing state T, so for any policy that sells at x,

$$J(x) = x,$$

exactly. And waiting collects nothing and lands on the next offer, whose distribution does not depend on the current one, so the value of waiting is the same number θ at every state, including N:

$$\theta = \alpha\, \mathbb{E}[J(w)] = \alpha \left(p_0 \theta + \sum_{i=1}^{5} p_i J(c_i) \right).$$

A policy is therefore described by the set of offers it accepts, and evaluating it means solving one scalar linear equation for θ. If the policy accepts the offers in a set S and waits on the rest,

$$\theta = \alpha \left(\Big(p_0 + \sum_{i \notin S} p_i\Big)\theta + \sum_{i \in S} p_i c_i \right) \quad \Longrightarrow \quad \theta = \frac{\alpha \sum_{i \in S} p_i c_i}{1 - \alpha(p_0 + \sum_{i \notin S} p_i)}.$$

Improvement is equally simple: accept c_i when $c_i \geq \theta$, wait otherwise. So policy iteration here is a scalar fixed-point iteration on a threshold.

The loop. Start from μ^0 = "never sell", whose value is $\theta^0 = 0$: a car that is never sold earns nothing.

Round 1 — improvement. Against $\theta^0 = 0$ every offer clears the threshold, so μ^1 sells at the first offer, $S = \{c_1, \ldots, c_5\}$.

Round 1 — evaluation. With $\sum_i p_i c_i = 0.05(10000) + 0.2(11000) + 0.3(12000) + 0.3(13000) + 0.1(14000) = 11600$ and nothing waited on,

$$\theta^1 = \frac{0.99(11600)}{1 - 0.99(0.05)} = \frac{11484}{0.9505} = 12082.06.$$

The jump from 0 is the whole difference between a policy that sells and one that does not.

Round 2 — improvement. Now 12082.06 beats $c_1 = 10000$, $c_2 = 11000$ and $c_3 = 12000$, so μ^2 waits on those three and sells only at 13000 or 14000: $S = \{c_4, c_5\}$.
Round 2 — evaluation. With $\sum_{i \in S} p_i c_i = 0.3(13000) + 0.1(14000) = 5300$ and waiting mass $p_0 + p_1 + p_2 + p_3 = 0.6$,

$$\theta^2 = \frac{0.99(5300)}{1 - 0.99(0.6)} = \frac{5247}{0.406} = 12923.65.$$

Round 3 — improvement. Against 12923.65 the offer $c_3 = 12000$ still does not clear, and $c_4 = 13000$ still does, so the accepted set is again $\{c_4, c_5\}$. The improvement step returned its input, so the policy is optimal:

$$\mu^\star(x) = \begin{cases} 1 \text{ (sell)} & \text{if } x \geq 13000 \\ 0 \text{ (wait)} & \text{otherwise,} \end{cases} \qquad \theta^\star = 12923.65.$$

Two evaluations and three improvements. As a check, the Bellman equation $\theta = \alpha\, \mathbb{E}[\max\{\theta, w\}]$ holds at $\theta^\star$ to machine precision.
What the answer says. The optimal reservation price 12923.65 is *above* the mean offer of 11600 (and above the mean conditional on there being an offer, 12210.53) — a seller with $\alpha = 0.99$ can afford to wait for the top two of five possible offers, because waiting a month costs only 1% of the value and 40% of offers clear the bar. It is also, note, not equal to any offer: the threshold is a value, and the policy it induces is a partition of the five offers into accept and wait.
Sensitivity to the two choices. The threshold depends on α exactly as patience should:

Table 7: The optimal threshold as a function of the monthly depreciation rate.

α	0.99	0.95	0.9	0.8
sell at or above	13000	12000	11000	10000
$\theta^\star$	12923.65	11825.17	10978.02	9666.67

At $\alpha = 0.8$ depreciation is severe enough that the first offer should be taken. Repairing the data the other way — keeping $p_0 = 0.01$ and rescaling $p_1, \ldots, p_5$ to sum to 0.99 — changes nothing at $\alpha = 0.99$ (threshold 13000, $\theta^\star = 12936.52$), though at $\alpha = 0.9$ the two repairs do straddle a boundary (11000 against 12000), which is a fair warning about how little slack a threshold policy has near a tie.
Finally, note that α plays two roles at once and that this is the trick of the formulation: it is both the monthly depreciation rate and the discount factor. Rescaling the offer by α^{-k} turns a shrinking asset with undiscounted value into a stationary offer distribution with a discounted objective, which is what makes a single state-independent θ possible.

Problem 11.11 Suppose we would like to estimate the average height of a Mechanical Engineering TU/e student from N samples. The percentage of female students is significantly smaller, say 20%, so the average height is

$$0.2 \times H_1 + 0.8 \times H_2,$$

where H_1 and H_2 are the (unknown) average heights of female and male students. Both height distributions are discretised with the same number of bins n, so that $h_1, \ldots, h_n$ and $g_1, \ldots, g_n$ are the possible heights of females and males, and assume all the h_i are equally likely and all the g_i are equally likely.

(i) Suppose one takes $N = 10$ randomly picked male and female students with male probability α. With $\alpha = 0.5$ and the data in the table, what are the averages with and without importance sampling?

ℓ	1	2	3	4	5	6	7	8	9	10
M or F	F	M	M	F	F	M	F	F	F	M
height	1.80	2.01	1.90	1.60	1.91	1.94	1.65	1.71	1.59	2.10

(ii) Suppose there were two additional male students in the table, of heights 1.90 and 1.91. Recompute both averages and discuss the values obtained in (i) and (ii).

i Solution

(i) Without importance sampling, average the ten heights:

$$\frac{1.80 + 2.01 + 1.90 + 1.60 + 1.91 + 1.94 + 1.65 + 1.71 + 1.59 + 2.10}{10} = 1.8210.$$

For the importance sampling estimate, note first what the two distributions are. The behaviour policy picks a sex with probability 0.5 each and then a bin uniformly, so each particular height is drawn with probability $0.5/n$. The target distribution assigns $0.2/n$ to each female height and $0.8/n$ to each male height. The importance ratios are therefore

$$\frac{0.2/n}{0.5/n} = \frac{2}{5} \quad \text{for a female sample,} \qquad \frac{0.8/n}{0.5/n} = \frac{8}{5} \quad \text{for a male one,}$$

independent of n and of the bin values, as the problem's footnote promises. The data has six females summing to 10.26 and four males summing to 7.95, so

$$\frac{1}{10}\Big(\tfrac{2}{5}(10.26) + \tfrac{8}{5}(7.95)\Big) = \frac{16.824}{10} = 1.6824.$$

(ii) With the two extra males the plain average becomes $22.02/12 = 1.8350$, and the importance sampling estimate

$$\frac{1}{12}\Big(\tfrac{2}{5}(10.26) + \tfrac{8}{5}(11.76)\Big) = \frac{22.92}{12} = 1.9100.$$

Discussion. The estimate in (i), 1.6824, is far below anything one would expect: the sample contains no height under 1.59 and the target population is 80% male, yet the estimate comes out below the female average of 1.71. In (ii) the estimate is 1.9100, which is sensible. Two samples of the same kind, one badly wrong.

The cause is visible in the sum of the weights, which ought to be N:

$$\text{(i)} \quad \tfrac{2}{5}(6) + \tfrac{8}{5}(4) = 8.8 \quad \text{against} \quad N = 10, \qquad \text{(ii)} \quad \tfrac{2}{5}(6) + \tfrac{8}{5}(6) = 12 \quad \text{against} \quad N = 12.$$

In (ii) the sample happens to contain exactly six females and six males, matching the behaviour probabilities $(0.5, 0.5)$, so the weights sum to N exactly and the estimator is on target. In (i) the sample is 6–4, the weights fall 12% short of N, and dividing by N scales the whole estimate down by that factor. Nothing is wrong with the weights; the problem is the normaliser.

Dividing by the sum of the weights instead — **weighted** importance sampling — repairs it:

$$\text{(i)} \quad \frac{16.824}{8.8} = 1.9118, \qquad \text{(ii)} \quad \frac{22.92}{12} = 1.9100.$$

The weighted estimate from the *ten*-sample data is already 1.9118, essentially the answer the twelve-sample data gives, and within 0.02 of the exact mixture of the sample means, $0.2(1.7100)+0.8(1.9875) = 1.9320$. In (ii) the two estimators coincide, because there the weights already summed to N.

So the lesson of (i) versus (ii) is sharper than "importance sampling has high variance". The variance is real, but here it enters through one specific channel — the random number of male and female draws — and normalising by the realised weights removes exactly that channel while leaving the reweighting intact. It is the same mechanism as in Problem 11.7 and the same one that separates the two off-policy estimators in `A_11_BlackjackOnOffPolicy.ipynb`.